

データ構造設計：ローカル永続化 ToDoリスト

目的と対象読者

このドキュメントは、ローカルストレージに永続化するToDoリストアプリケーションのデータ構造設計について解説します。主な対象読者は、JavaScriptと基本的なWeb開発の知識を持つ開発者で、特にlocalStorageやIndexedDBなどのブラウザストレージ技術を利用したアプリケーション開発に関心のある方です。

この資料を読むことで、以下の点を理解できます。

- ToDoリストアプリケーションに適したデータ構造の選択肢とその理由。
- 基本的なデータ構造（配列、オブジェクト）の特性と、それらを組み合わせた設計方法。
- データ操作（追加、読み取り、更新、削除）の効率性を考慮した設計のポイント。
- パフォーマンスや保守性に関する考慮事項。

1. はじめに：なぜデータ構造設計が重要か？

ToDoリストのようなシンプルなアプリケーションでも、データ構造の設計は非常に重要です。適切なデータ構造を選択することで、以下のようなメリットが得られます。

- 効率的なデータ操作**：タスクの追加、検索、更新、削除がスムーズに行えます。
- コードの可読性と保守性の向上**：データが整理されていると、コードが理解しやすくなり、将来的な機能追加や修正も容易になります。
- パフォーマンスの最適化**：特にデータ量が増えた場合に、不適切なデータ構造はパフォーマンスのボトルネックとなり得ます。
- 拡張性の確保**：将来的に新しい機能（例：優先度、タグ付け）を追加する際に、柔軟に対応できます。

このドキュメントでは、主に `localStorage` を利用するケースを想定し、シンプルかつ効果的なデータ構造を提案します。

2. ToDoリストの基本要素

まず、ToDoリストのタスク（ToDoアイテム）が持つべき基本的な情報を定義します。

- **ID**：各タスクを一意に識別するためのID（例：`Date.now()` やUUID）。
- **内容（content）**：タスクの具体的な説明（例：“牛乳を買う”）。
- **完了状態（completed）**：タスクが完了したかどうかを示す真偽値（`true` / `false`）。
- **作成日時（createdAt）**：タスクが作成された日時。
- **更新日時（updatedAt）**：タスクが最後に更新された日時。

将来的には、期日、優先度、タグなどの要素を追加することも考えられます。

3. データ構造の選択肢と設計

ToDoリスト全体のデータと、個々のToDoアイテムのデータをどのように表現するかを考えます。

3.1. 個々のToDoアイテムの表現

個々のToDoアイテムは、複数の情報（ID、内容、完了状態など）を持つため、JavaScriptの **オブジェクト** を使用するのが最も自然で効率的です。

```
// ToDoアイテムの例
const todoItem = {
  id: Date.now(), // 一意のID
  content: "演習資料のレビュー",
  completed: false,
  createdAt: new Date().toISOString(),
  updatedAt: new Date().toISOString()
};
```

3.2. ToDoリスト全体の表現

複数のToDoアイテムを管理するためには、JavaScriptの **配列** を使用するのが一般的です。配列の各要素が、前述のToDoアイテムオブジェクトとなります。

```
// ToDoリスト全体のデータ構造例
let todos = [
  { id: 1678886400000, content: "設計書作成", completed: true, createdAt: "...", updatedAt: "..."},
  { id: 1678886400001, content: "コーディング", completed: false, createdAt: "...", updatedAt: "..."},
];
```

```
{ id: 1678886400002, content: "テスト", completed: false, createdAt: "...", updatedAt: "..." }
];
```

この「オブジェクトの配列」という形式は、`JSON.stringify()` を使って文字列化しやすく、`localStorage` に保存するのに適しています。

3.3. なぜこの構造か？（配列 + オブジェクト）

- **直感的で理解しやすい**：多くの開発者にとって馴染み深い構造です。
- **JavaScriptの標準メソッドとの親和性**：配列メソッド（`map` , `filter` , `find` , `findIndex` , `push` , `splice` など）を活用して、データの検索、追加、削除、更新が容易に行えます。
- **JSONとの互換性**：`localStorage` は文字列しか保存できないため、`JSON.stringify()` で文字列に変換し、`JSON.parse()` でオブジェクトに戻す操作が必須です。この構造はJSONとの相性が抜群です。
- **順序保持**：配列は要素の順序を保持するため、ユーザーがタスクを追加した順序や、意図的に並び替えた順序を維持できます。

4. CRUD操作とデータ構造

このデータ構造（オブジェクトの配列）を使って、基本的なCRUD操作（作成、読み取り、更新、削除）をどのように行うか見ていきましょう。

4.1. 作成（Create）

新しいToDoアイテムをリストに追加します。

```
function addTodo(content) {
  const newTodo = {
    id: Date.now(),
    content: content,
    completed: false,
    createdAt: new Date().toISOString(),
    updatedAt: new Date().toISOString()
  };
  todos.push(newTodo); // 配列の末尾に追加
  saveTodos(); // localStorageに保存する関数（後述）
  return newTodo;
}
```

4.2. 読み取り（Read）

- **全件取得**：`todos` 配列全体を返します。

- **IDによる個別取得** : `find` メソッドを使用します。

```
function getAllTodos() {
  return todos;
}

function getTodoById(id) {
  return todos.find(todo => todo.id === id);
}
```

4.3. 更新 (Update)

特定のToDoアイテムの内容や完了状態を変更します。 `findIndex` で対象のインデックスを見つけ、直接プロパティを更新します。

```
function updateTodoContent(id, newContent) {
  const todoIndex = todos.findIndex(todo => todo.id === id);
  if (todoIndex > -1) {
    todos[todoIndex].content = newContent;
    todos[todoIndex].updatedAt = new Date().toISOString();
    saveTodos();
    return todos[todoIndex];
  }
  return null; // 見つからなかった場合
}

function toggleTodoCompleted(id) {
  const todoIndex = todos.findIndex(todo => todo.id === id);
  if (todoIndex > -1) {
    todos[todoIndex].completed = !todos[todoIndex].completed;
    todos[todoIndex].updatedAt = new Date().toISOString();
    saveTodos();
    return todos[todoIndex];
  }
  return null;
}
```

4.4. 削除 (Delete)

特定のToDoアイテムをリストから削除します。 `filter` メソッドや `splice` メソッドを使用できます。

```
// filter を使用する場合（非破壊的だが、新しい配列が生成される）
function deleteTodo(id) {
  const initialLength = todos.length;
```

```
todos = todos.filter(todo => todo.id !== id);
if (todos.length < initialLength) {
  saveTodos();
  return true; // 削除成功
}
return false; // 対象が見つからず削除失敗
}

// splice を使用する場合（破壊的だが、効率的な場合がある）
function deleteTodoWithSplice(id) {
  const todoIndex = todos.findIndex(todo => todo.id === id);
  if (todoIndex > -1) {
    todos.splice(todoIndex, 1); // todoIndexから1要素削除
    saveTodos();
    return true;
  }
  return false;
}
```

4.5. localStorageへの保存と読み込み

`localStorage` には文字列として保存するため、`JSON.stringify` と `JSON.parse` を使用します。

```
const STORAGE_KEY = 'todos-app-data';

// todos配列をlocalStorageに保存
function saveTodos() {
  try {
    localStorage.setItem(STORAGE_KEY, JSON.stringify(todos));
  } catch (e) {
    console.error("localStorageへの保存に失敗しました。", e);
    // ここでユーザーにエラーを通知するなどの処理を追加できます
  }
}

// localStorageからtodos配列を読み込む
function loadTodos() {
  try {
    const storedTodos = localStorage.getItem(STORAGE_KEY);
    if (storedTodos) {
      todos = JSON.parse(storedTodos);
    } else {
      todos = []; // 保存されているデータがない場合は空の配列で初期化
    }
  } catch (e) {
    console.error("localStorageからの読み込みに失敗しました。", e);
    todos = []; // エラー時も空の配列で初期化
    // ユーザーにエラーを通知するなどの処理
  }
  return todos;
}
```

```
}  
  
// アプリケーション初期化時に読み込み  
loadTodos();
```

エラーハンドリングの注意点:`JSON.parse` は不正なJSON文字列をパー

スしようとするエラーをスローします。また、`localStorage.setItem` も
ストレージ容量の制限などで失敗する可能性があります。そのため、

`try...catch`ブロックで囲むことが推奨されます。詳細は「[補足資料: try-](#)[catch詳解](#)」や「[補足資料: JSON操作詳解](#)」を参照してください。

5. パフォーマンスに関する考慮

現状の「オブジェクトの配列」構造は、数百～数千程度のToDoアイテムであれば、ほとんどの現代のブラウザで問題なく動作します。しかし、データ量が非常に多くなる（数万件以上）可能性がある場合、以下の点を考慮する必要が出てくるかもしれません。

- **検索効率** : ID以外の条件（例：内容の部分一致検索）で頻繁に検索する場合、配列の線形検索（`find` , `filter`）はデータ量に比例して遅くなります。
- **更新効率** : 特定のアイテムを更新する際も、まず `findIndex` で検索するコストがかかります。

大規模データへの対策案（参考）:

- **Mapオブジェクトの利用** : ToDoアイテムのIDをキーとし、ToDoオブジェクトを値とする `Map` を使用すると、IDによる検索・更新・削除が `O(1)` に近くなり高速です。ただし、順序の保持には工夫が必要になります（例：別途IDの配列を保持する）。

```
// Map を使った場合のイメージ  
let todosMap = new Map(); // { id1 => todoObj1, id2 => todoObj2 }  
let todoOrder = []; // [id1, id2, ...] (順序保持用)
```

- **IndexedDBの利用** : より複雑なクエリや大量のデータを扱う場合は、ブラウザの本格的なデータベースである `IndexedDB` の利用を検討します。`IndexedDB` はインデックスを利用した高速な検索が可能です。詳細は「[補足資料: ブラウザストレージ比較](#)」を参照してください。

ただし、本演習の範囲では、シンプルな「オブジェクトの配列」構造で十分対応可能です。

6. 代替案との比較

データ構造	メリット	デメリット	主な用途/考察
オブジェクトの配列（推奨）	直感的、JSON互換性高い、配列メソッド活用可、順序保持	大量データ時の検索/更新効率が低下する可能性	小～中規模のデータセット、localStorageとの連携が容易。本演習ではこの構造を採用。
IDをキーとするオブジェクト	IDによるアクセスが高速（ <code>O(1)</code> ）	順序保持が困難、 <code>Object.values()</code> などで配列化しないとイテレーションしにくい	IDベースのアクセスが主で、順序が重要でない場合。localStorageに保存する際は結局配列化か、キーの配列を別途持つ必要がある。
Mapオブジェクト	IDによるアクセスが高速（ <code>O(1)</code> ）、キーと値に任意の型を使用可、順序保持（挿入順）	localStorageに直接保存不可（シリアライズ/デシリアライズの工夫が必要）	IDベースのアクセスが多く、挿入順が重要な場合に有効。シリアライズ方法（例： <code>Array.from(map.entries())</code> ）を定義すればlocalStorageでも利用可能。
複数の配列	（特定のケースでメモリ効率が良い場合があるが、一般的ではない）	データの一貫性維持が複雑、コードが煩雑になりやすい	通常のアプリケーション開発では推奨されない。
正規化されたデータ構造	データ重複の削減、一貫性の向上（リレーショナルデータベースの考え方）	構造が複雑化、データの取得や更新に複数ステップが必要になる場合がある	関連するデータが多い複雑なアプリケーション（例：プロジェクト管理ツールなど）。IndexedDBと組み合わせることが多い。

本演習では、シンプルさ、`localStorage` との親和性、JavaScriptの標準的な操作との整合性を考慮し、「オブジェクトの配列」を採用します。

7. 図解によるデータ構造のイメージ

```
graph TD
    A[localStorage] -- JSON.stringify / JSON.parse --> B(ToDoリスト 配列: todos);
    B -- contains --> C1{ToDoアイテム1 オブジェクト};
    B -- contains --> C2{ToDoアイテム2 オブジェクト};
    B -- contains --> C3{...};

    C1 --> ID1[id: 数値/文字列];
    C1 --> Content1[content: 文字列];
    C1 --> Completed1[completed: 真偽値];
    C1 --> CreatedAt1[createdAt: 文字列(ISO形式)];
```

```
C1 --> UpdatedAt1[updatedAt: 文字列(IS0形式)];

C2 --> ID2[id: 数値/文字列];
C2 --> Content2[content: 文字列];
C2 --> Completed2[completed: 真偽値];
C2 --> CreatedAt2[createdAt: 文字列(IS0形式)];
C2 --> UpdatedAt2[updatedAt: 文字列(IS0形式)];

subgraph "ToDoアイテムの構造 (例)"
  direction LR
  D_id[id]
  D_content[content]
  D_completed[completed]
  D_createdAt[createdAt]
  D_updatedAt[updatedAt]
end
```

上の図は、`localStorage` に保存されるToDoリストのデータ構造の概念を示しています。`todos` というキーで、ToDoアイテムのオブジェクトが格納された配列がJSON文字列として保存されます。各ToDoアイテムオブジェクトは、`id`、`content`、`completed`、`createdAt`、`updatedAt`といったプロパティを持ちます。

8. まとめとベストプラクティス

- **シンプルさを保つ**：特に初期段階では、最もシンプルで理解しやすいデータ構造から始めるのが良いでしょう。
- **一貫性のある命名**：プロパティ名（`id`、`content`、`completed` など）は一貫性を持たせましょう。
- **不変性（Immutability）の考慮**：データを更新する際に、元の配列やオブジェクトを直接変更するのではなく、新しい配列やオブジェクトを生成するアプローチ（特にReactやVue.jsなどのフレームワークと組み合わせる場合）も検討できますが、本演習のシンプルなCRUD操作では直接変更でも問題ありません。`filter` は新しい配列を返りますが、`splice` やプロパティの直接代入は元のデータを変更します。
- **IDの重要性**：各アイテムを一意に識別するためのIDは不可欠です。`Date.now()`、`crypto.randomUUID()`（モダンブラウザで利用可能）、あるいは単純な連番など、適切な方法で生成します。
- **エラーハンドリング**：`localStorage` へのアクセスや `JSON` のパース時には、エラーが発生する可能性があるため、`try...catch` を適切に使用します。

このドキュメントで提案した「オブジェクトの配列」というデータ構造は、多くのToDoリストアプリケーションにとって堅牢で扱いやすい出発点となります。

9. 関連する補足資料

- [JSON操作詳解](#)：`JSON.stringify()` と `JSON.parse()` の詳細な使い方、注意点について。

- [localStorage API詳解](#): localStorage の基本的な使い方、制限、注意点について。
 - [try-catch詳解](#): JavaScriptにおけるエラーハンドリングの基本について。
 - [JavaScript オブジェクト詳解](#): JavaScriptのオブジェクトのより詳細な操作方法について。
 - [ブラウザストレージ比較](#): localStorage 以外のブラウザストレージ技術 (sessionStorage , IndexedDB など) との比較。
-

最終更新日: 2024/06/13